

State & Events in React

Part A — PPT Content: 12 Slides

Learning Objectives

By the end of this hour, students will understand:

What is state

Difference between props and state

useState() hook

Updating state correctly

How state causes re-rendering

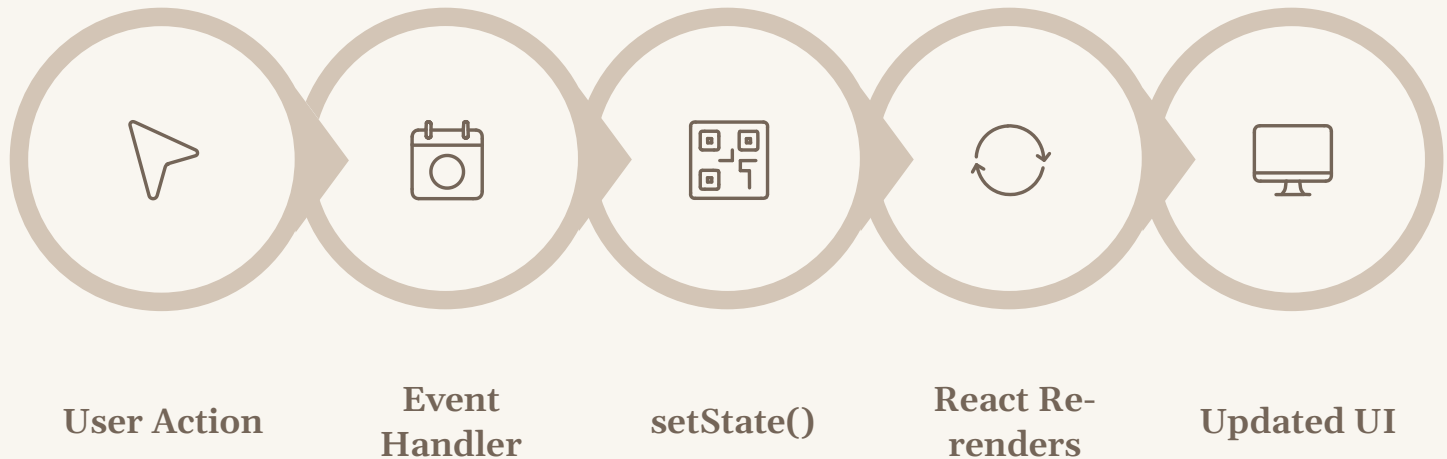
Handling click events

Handling input events

Controlled components

Common event-handling mistakes

Core Flow



Slide 2 — Props vs State

Props

Data received from a parent.

```
<StudentCard name="Aditi" />
```

- External to the component
- Read-only
- Passed parent → child

State

Data owned by the component.

```
const [count, setCount] = useState(0);
```

- Internal to the component
- Can change
- Changing state causes re-render

Simple Comparison

Props	State
Received from parent	Owned by component
Read-only	Updated using setter
Input to component	Internal data
Parent controls value	Component controls value

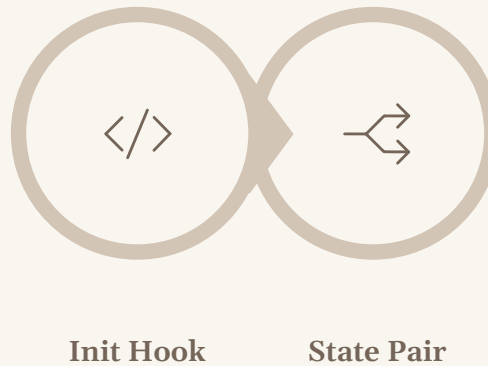
Slide 3 — What is useState()?

useState allows a function component to remember information between renders.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <p>Count: {count}</p>
  );
}
```

Think of:



So:

```
const [count, setCount] = useState(0);
```

means:

count
current value

setCount
function to change value

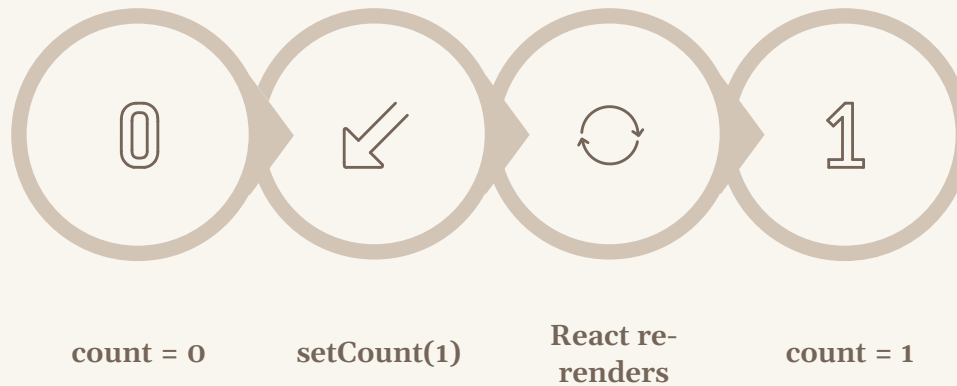
0
initial value

Slide 4 — Updating State

Example:

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return (  
    <div>  
      <p>Students: {count}</p>  
      <button onClick={() => setCount(count + 1)}>  
        Add Student  
      </button>  
    </div>  
  );  
}
```

When the button is clicked:



Important

Never do:

```
count++;
```

Instead:

```
setCount(count + 1);
```

Slide 5 — State is Local

State belongs to a **component instance**.

If we render:

```
<Counter />  
<Counter />
```

React creates two independent states.

Counter #1

count = 5

Counter #2

count = 2

Changing Counter #1 does not change Counter #2.

Key idea

📌 State is local unless you deliberately move/share it through another component.

Slide 6 — What are React Events?

React handles user interactions using event props.

Common events:

onClick

onChange

onSubmit

onMouseEnter

onFocus

onBlur

Example:

```
<button onClick={handleClick}>  
  Add Student  
</button>
```

React event names use **camelCase**.

✓ **onClick**

✗ **not: onclick**

Slide 7 — Event Handler: Function Reference

Correct:

```
<button onClick={handleSearch}>  
  Search  
</button>
```

React calls the function when the user clicks.

Common fresher mistake

```
<button onClick={handleSearch()}>
```

This calls the function **during rendering**.

Remember

✓ Function reference

```
onClick={handleSearch}  
↑  
function reference
```

✗ Not function execution

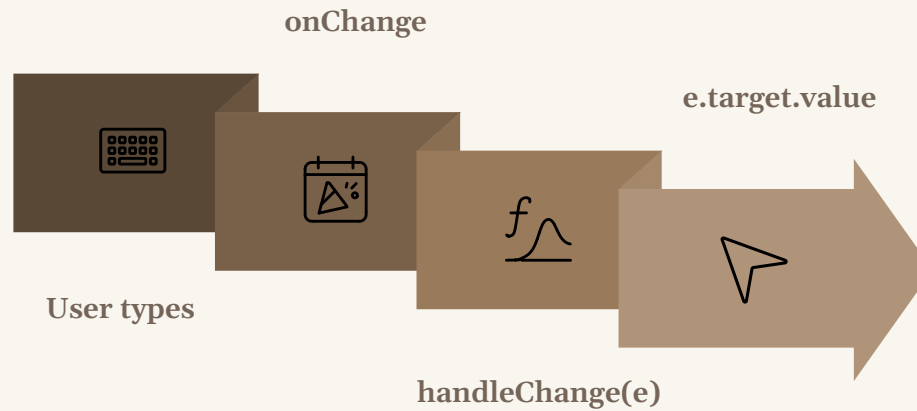
```
onClick={handleSearch()}  
↑  
function execution
```

Slide 8 — Handling Input Events

Example:

```
function SearchBox() {  
  const [query, setQuery] = useState("");  
  
  function handleChange(e) {  
    setQuery(e.target.value);  
  }  
  
  return (  
    <input value={query} onChange={handleChange} />  
  );  
}
```

When the user types:



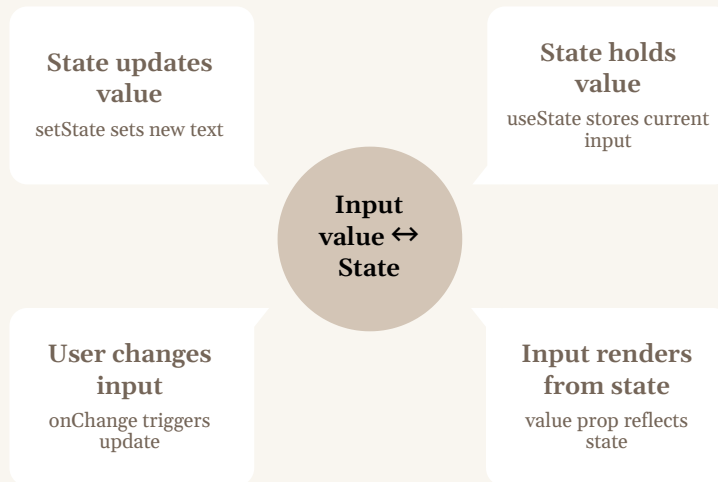
Slide 9 — Controlled Components

A controlled input gets its value from React state.

```
const [query, setQuery] = useState("");

<input
  value={query}
  onChange={(e) => setQuery(e.target.value)}
/>
```

React controls:



Why use controlled inputs?

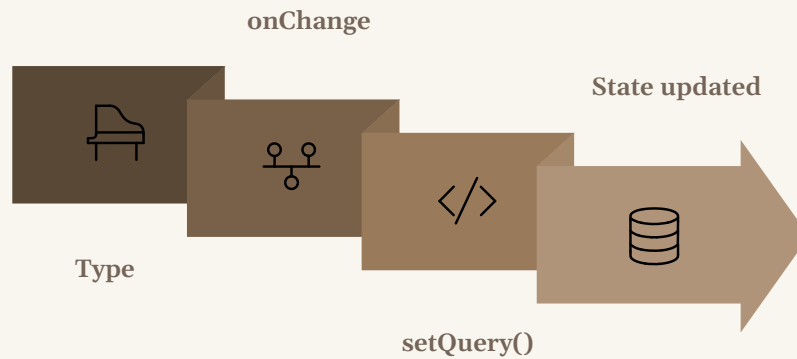
- React always knows the current value
- Easy validation
- Easy search/filtering
- Easy form submission
- UI stays synchronized with state

Slide 10 — Combining State + Events

Example Student Search:

```
function SearchBox() {  
  const [query, setQuery] = useState("");  
  
  function handleSearch() {  
    console.log("Searching for:", query);  
  }  
  
  return (  
    <div>  
      <input  
        value={query}  
        onChange={(e) => setQuery(e.target.value)}  
      />  
      <button onClick={handleSearch}>  
        Search  
      </button>  
    </div>  
  );  
}
```

Flow



Slide 11 — Common Fresher Mistakes

Mistake 1 — Direct state mutation

 Wrong

```
count++;
```

 Correct

```
setCount(count + 1);
```

Mistake 2 — Calling event handler immediately

 Wrong

```
onClick={handleClick()}
```

 Correct

```
onClick={handleClick}
```

Mistake 3 — Forgetting value

```
<input onChange={handleChange} />
```

For a controlled input:

```
<input value={query} onChange={handleChange} />
```

Mistake 4 — Wrong event name

 Wrong

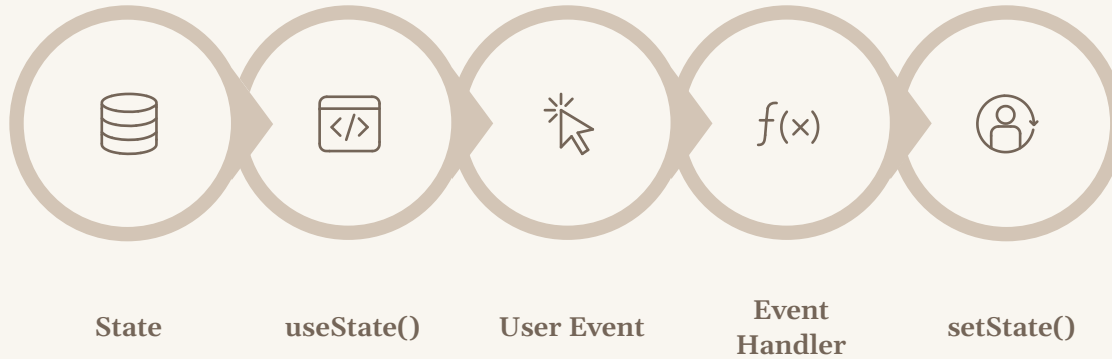
```
onclick
```

 Correct

```
onClick
```

Slide 12 — Key Takeaways

Remember



Students should now be able to:

- Create state with `useState`
- Update state using setter functions
- Handle button clicks
- Handle input changes
- Create controlled inputs
- Explain props vs state
- Avoid common event-handling mistakes

Next Hour

- Conditional Rendering & Lists